

# Implementación chatbot inteligente con Azure OpenAI

María Cruz Arcos Ciórraga

**Programa Avanzado en Inteligencia Artificial para Programar**

15/05/2026

|                                                 |    |
|-------------------------------------------------|----|
| 1. Introducción.....                            | 1  |
| 2. Objetivos .....                              | 2  |
| 3. Configuración entorno Azure .....            | 3  |
| 4. Preparación entorno desarrollo.....          | 4  |
| 5. Desarrollo del chatbot.....                  | 5  |
| 6. Pruebas realizadas .....                     | 6  |
| Caso de uso 1: Curiosidad sobre ciencia.....    | 7  |
| Caso de uso 2: Respuesta técnica .....          | 7  |
| Caso de uso 3: Generación canción infantil..... | 8  |
| 7. Análisis de costos y optimización.....       | 8  |
| Análisis .....                                  | 9  |
| Optimización .....                              | 10 |
| 8. Conclusiones .....                           | 10 |

# 1. Introducción

En esta actividad se desarrolla un chatbot inteligente utilizando Python y la API de Azure OpenAI.

El objetivo principal es implementar una aplicación capaz de interactuar con el usuario mediante un bucle conversacional, generando respuestas automáticas basadas en un modelo desplegado en Azure.

Además, se busca comprender el impacto de los parámetros de configuración del modelo (como temperature, max\_tokens o top\_p) y analizar el consumo de tokens para optimizar costes.

## 2. Objetivos

Los objetivos principales de esta actividad son:

- Configurar un recurso de Azure OpenAI en el portal de Azure y obtener credenciales necesarias
- Preparar un entorno de desarrollo en Python.
- Implementar un chatbot funcional en un archivo ``chatbot.py``.
- Probar el funcionamiento del chatbot con diferentes casos de uso.
- Analizar costes aproximados y proponer estrategias de optimización.

### 3. Configuración entorno Azure

- Creación de recurso Azure OpenAI

|                           |                                        |                    |                                                                   |
|---------------------------|----------------------------------------|--------------------|-------------------------------------------------------------------|
| ^ Información esencial    |                                        | Tipo de API        | : OpenAI                                                          |
| Grupo de recursos (mover) | : <a href="#">GrupoUNIR</a>            | Plan de tarifa     | : Estándar                                                        |
| Estado                    | : Activo                               | Puntos de conexión | : <a href="#">Hacer clic aquí para ver los puntos de conexión</a> |
| Ubicación                 | : Germany West Central                 | Administrar claves | : <a href="#">Haga clic aquí para administrar claves</a>          |
| Suscripción (mover)       | : <a href="#">Azure for Students</a>   |                    |                                                                   |
| Id. de suscripción        | : 2e47d153-986d-4c4d-8273-8b52fda21d5e |                    |                                                                   |
| Etiquetas (editar)        | : <a href="#">Agregar etiquetas</a>    |                    |                                                                   |

- Despliegue del modelo

**Punto de conexión**

URI de destino

https://act2-openai.openai.azure.com/openai/responses?api-version=20... 

Clave

..... 



**Información de implementación**

|                        |                   |                                         |                                                              |
|------------------------|-------------------|-----------------------------------------|--------------------------------------------------------------|
| Nombre                 | gpt-5.4-nano      | Estado de aprovisionamiento             | Correcto                                                     |
| Tipo de implementación | Estándar global   | Creado el                               | 2026-05-14T17:50:28.206759Z                                  |
| Creado por             | mcac919@gmail.com | Modificado el                           | May 14, 2026 7:50 PM                                         |
| Modificado por         | mcac919@gmail.com | Directiva de actualización de versiones | Una vez que haya disponible una nueva versión predeterminada |

|                                         |         |                                              |     |
|-----------------------------------------|---------|----------------------------------------------|-----|
| Límite de velocidad (tokens por minuto) | 250.000 | Límite de velocidad (solicitudes por minuto) | 250 |
|-----------------------------------------|---------|----------------------------------------------|-----|

|                          |                      |                              |                      |
|--------------------------|----------------------|------------------------------|----------------------|
| Nombre del modelo        | gpt-5.4-nano         | Versión del modelo           | 2026-03-17           |
| Estado del ciclo de vida | GenerallyAvailable   | Fecha de creación            | Mar 17, 2026 1:00 AM |
| Fecha de actualización   | Mar 17, 2026 1:00 AM | Fecha de retirada del modelo | Mar 18, 2027 1:00 AM |

- Obtención de credenciales (API key y endpoint)

Clave 1

..... 

Clave 2

..... 

Ubicación o región ⓘ

germanywestcentral 

Extremo

https://act2-openai.openai.azure.com/ 

## 4. Preparación entorno desarrollo

- Python 3.14.3
- Visual Studio Code
- Creación y activación del entorno virtual para aislar las dependencias del proyecto

```
python -m venv venv
```

- Instalación de dependencias

```
pip install openai
```

## 5. Desarrollo del chatbot

El chatbot implementado funciona mediante un bucle conversacional:

1. Solicita al usuario un mensaje.
2. Envía el mensaje a Azure OpenAI.
3. Recibe una respuesta generada por el modelo.
4. Muestra la respuesta por pantalla.
5. Repite el proceso hasta que el usuario escriba `"salir"`.

Para definir el comportamiento del modelo se han utilizado roles:

- `system`: define las reglas del asistente.
- `user`: mensajes del usuario.
- `assistant`: respuestas generadas.

Además, se han incluido parámetros configurables vía variables de entorno para controlar la generación:

- `temperature`: controla creatividad
- `max_tokens`: límite de tokens en la respuesta.
- `top_p`: controla muestreo probabilístico.
- `frequency_penalty`: reduce repeticiones.
- `presence_penalty`: incentiva temas nuevos.

*Ver fichero adjunto `chatbot.py`*

## 6. Pruebas realizadas

A continuación, se documentan tres escenarios de prueba con resultados y análisis:

### *Caso de uso 1: Curiosidad sobre ciencia*

Chatbot listo. Escribe 'salir' para terminar.

Tú: Cuentame alguna curiosidad sobre los ultimos descubrimientos en ciencia

Asistente: ¡Claro! Aquí va una curiosidad (bastante reciente y muy llamativa) sobre ciencia:

**\*\*Se están usando “relojes atómicos” ultrarrápidos y extremadamente precisos para estudiar si la física cambia con el tiempo... y hasta para buscar “señales” de materia oscura.\*\***

La idea es que algunos fenómenos en el universo podrían hacer que *\*constantes fundamentales\** (como ciertas propiedades relacionadas con la estructura atómica) varíen muy poquito, o que el “campo” de la materia oscura interactúe de forma sutil con la materia. Los relojes atómicos, al ser tan precisos, permiten medir cambios minúsculos en frecuencias que, en teoría, deberían permanecer estables.

**\*\*Lo curioso\*\*** es que no es solo “medir el tiempo”: estos aparatos se están convirtiendo en una especie de **\*\*detectores de física nueva\*\***. Si el universo tiene “ruido” o variaciones que antes no podíamos detectar, los relojes atómicos podrían ser una de las maneras de capturarlo.

Si quieres, dime qué te interesa más y te cuento una curiosidad enfocada:

1) medicina y biología, 2) espacio y astronomía, 3) física/IA, o 4) clima y geociencia.

**Coherencia:** alta. Explica una idea de física (relojes atómicos y posibles señales).

**Detalle:** correcto adaptándose a nivel divulgativo.

**Observación:** el asistente suena bastante seguro, pero no cita fuentes.



## Caso de uso 2: Respuesta técnica

```
Chatbot listo. Escribe 'salir' para terminar.

Tú: Cuales son las principales novedades en la última versión de c++
Asistente: Depende de qué versión exacta te refieras (C++20, C++23, o C++26 "aún en desarrollo"). A fecha de hoy, lo más "oficial y relevante" normalmente es C++23 (y C++26 ya está en marcha, pero con cambios que pueden variar según el estado del estándar/compilador).

Te resumo las principales novedades de C++23 (las más comunes en noticias y uso real):

## Novedades destacadas de C++23

### 1) Mejoras de std::expected y manejo de errores
- Se consolida el enfoque de errores como valores (en lugar de excepciones) con std::expected.
- Ayuda a escribir código más claro para "éxito/fracaso" sin tanto boilerplate.

### 2) std::mdspan (multi-dimensional spans)
- Similar a std::span, pero para vistas multidimensionales.
- Muy útil para librerías numéricas, ML, arrays de matrices, etc., porque permite "mirar" datos sin copiar y con layouts flexibles.

### 3) Mejoras en algoritmos y librería estándar
- Se agregan/ajustan algoritmos y utilidades que mejoran rendimiento y ergonomía.
- Muchas mejoras son "pequeñas pero útiles": más funciones constexpr, mejor soporte con ranges, etc.

### 4) Mejoras con std::ranges (C++20+)
- C++23 sigue puliendo el ecosistema de ranges: más adaptadores/overloads y casos más cubiertos.
- Resultado: menos iteradores "a mano" y más transformaciones declarativas.

### 5) Texto/Unicode y utilidades relacionadas (en la librería)
- Cambios y mejoras alrededor de soporte de caracteres y utilidades de texto (según implementación/compilador).
- No es tan "una feature única", sino más bien una evolución.

### 6) std::print / std::format (según implementación)
- En la práctica, lo que más se ve es la adopción de facilidades de formateo e impresión.
- Ojo: el soporte exacto depende del compilador y versión (GCC/Clang/MSVC).

---

## Importante: "última versión" en la práctica = soporte del compilador
Aunque el estándar diga una cosa, lo que puedes usar ya depende de:
- GCC (y su versión)
- Clang
- MSVC

Por eso, para afinar:
¿Qué compilador y versión usas (GCC/Clang/MSVC y número)?
Si me dices eso, te digo exactamente qué features de C++23 ya están disponibles y cuáles todavía no, y te paso ejemplos cortos.
```

**Coherencia:** muy buena. Primero pide aclaración (C++20/23/26) y luego resume.

**Detalle:** bastante completo a nivel técnico (`expected`, `mdspan`, `ranges`, etc.).

**Observación:** el asistente explica correctamente que "lo oficial" vs "lo disponible" depende del compilador, algo muy importante de remarcar.

## Caso de uso 3: Generación canción infantil

```
Chatbot listo. Escribe 'salir' para terminar.

Tú: Genera una canción infantil breve que tenga rima consonante y sea facil de recordar
Asistente: En la luna hay un gatito,
con un sombrero chiquito.
Miau miau, ¡qué bien cantó!,
y su risa me guiñó.
```

**Coherencia:** muy precisa. Canción infantil y breve.

**Observación:** el texto mantiene rima consonante de forma consistente, y es fácil de recordar.

## 7. Análisis de costos y optimización

Azure OpenAI factura en base al consumo de tokens, distinguiendo entre:

- \* Tokens de entrada (prompt)
- \* Tokens de salida (completion)

Para estimar el coste del servicio, se puede hacer uso de la herramienta Azure Pricing Calculator. Tras seleccionar el producto (Azure OpenAI), se deben definir los parámetros de configuración clave: región, tipo de modelo, modelo específico y estrategia de precios. Finalmente, al introducir una estimación del volumen de tokens de entrada y salida, la calculadora genera una proyección detallada de los costes mensuales y anuales en función del uso previsto

Azure OpenAI

Language Models, GPT-4o 128K, Standard (On ...

Upfront: \$0.00

Monthly: \$0.00

Annual: \$0.00

Azure OpenAI

[Give feedback](#)

Region:

East US

Model type:

Language Models

Model:

GPT-4o 128K

Pricing Strategy:

Standard (On Demand)

Language Models

Use this link to check [service availability](#).

Deployment type:

Global

Input

0

x

\$0.00500

=

\$0.00

x1,000 tokens

Per 1,000 tokens

Cached Input

Cached Input is not available for the selected model

Output

0

x

\$0.01500

=

\$0.00

x1,000 tokens

Per 1,000 tokens

Upfront cost

\$0.00

Monthly cost

\$0.00

Annual cost

\$0.00

Support

SUPPORT:

Basic (Included)

\$0.00

Select your program/offer

LICENSING PROGRAM:

Microsoft Customer Agreement (MCA)

[Log in](#) to see your Azure agreement pricing.

Show Dev/Test Pricing

Estimated upfront cost

\$0.00

Estimated monthly cost

\$0.00

Estimated annual cost

\$0.00

Export

Save

Share

[Log in](#) to save and share cost estimates.

CURRENCY

United States - Dollar (\$) USD

## *Análisis*

En nuestro código, se ha añadido al final de cada respuesta del asistente el consumo de tokens para realizar un análisis introduciendo diferentes peticiones en el chatbot.

Durante las pruebas se observó:

- Prompts cortos y claros: menos tokens
- Respuestas largas: más tokens
- Respuestas creativas: más tokens
- Historial largo: aumento progresivo del coste

## *Optimización*

Se proponen las siguientes estrategias:

- Limitar ``max_tokens`` para evitar respuestas excesivamente largas.
- Ajustar ``temperature`` según el uso para adecuar la creatividad al caso de uso.
- Reducir el historial enviado en cada interacción.
- Resumir conversaciones largas antes de enviarlas al modelo.
- Implementar cache de respuestas repetidas
- Usar modelos más económicos para ajustarse al costo deseado

## 8. Conclusiones

En esta actividad se logró implementar un chatbot funcional en Python conectado a Azure OpenAI.

Se comprobó que la configuración de parámetros influye directamente en la calidad de las respuestas, así como en el consumo de tokens y coste asociado.

Finalmente, se identificaron varias estrategias para reducir el gasto optimizando el uso de tokens.